

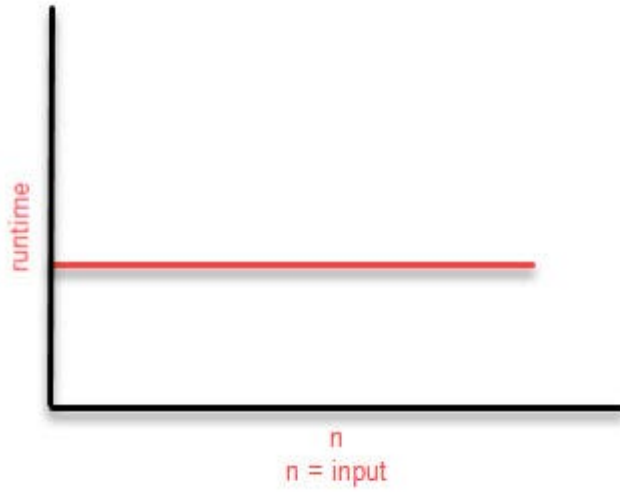


Quantum Algorithms Time Complexity

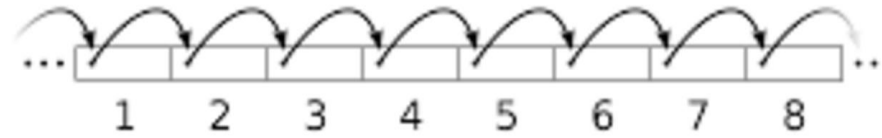
Algorithm	Problem Type	Time Complexity
 QAOA (Quantum Approximate Optimization Algorithm)	Combinatorial optimization (Max-Cut, QUBO)	$O(I \cdot M \cdot p \cdot E)$
 QUBO (via QAOA / VQE)	Quadratic optimization	$O(I \cdot M \cdot p \cdot n^2)$
 HHL (Harrow-Hassidim-Lloyd) Linear systems $Ax = b$	Linear systems $Ax = b$	$O\left(\frac{K^2 \log N}{\epsilon}\right)$ $K = \alpha$ condition number
 VQE (Variational Quantum Eigensolver)	Ground state energy estimation $A = \text{ansatz size}$	$O(I \cdot M \cdot A)$ ($A = \text{ansatz size}$)
 BB84 Quantum key distribution (KQD)	Quantum key distribution (QKD)	$O(n)$ or $O(\text{poly}(\log N))$
 QPE (Quantum Phase Estimation)	Eigenvalue / phase estimation	$O(t \cdot \text{poly}(\log N, 1/\epsilon))$
 Hamiltonian Simulation	Quantum system evolution	$O(t \cdot \text{poly}(\log N, 1/\epsilon))$

$O(1)$ — Constant Time

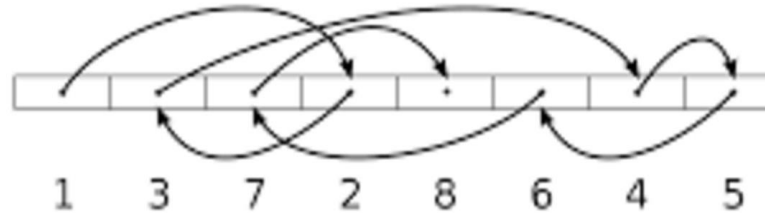
CONSTANT TIME COMPLEXITY



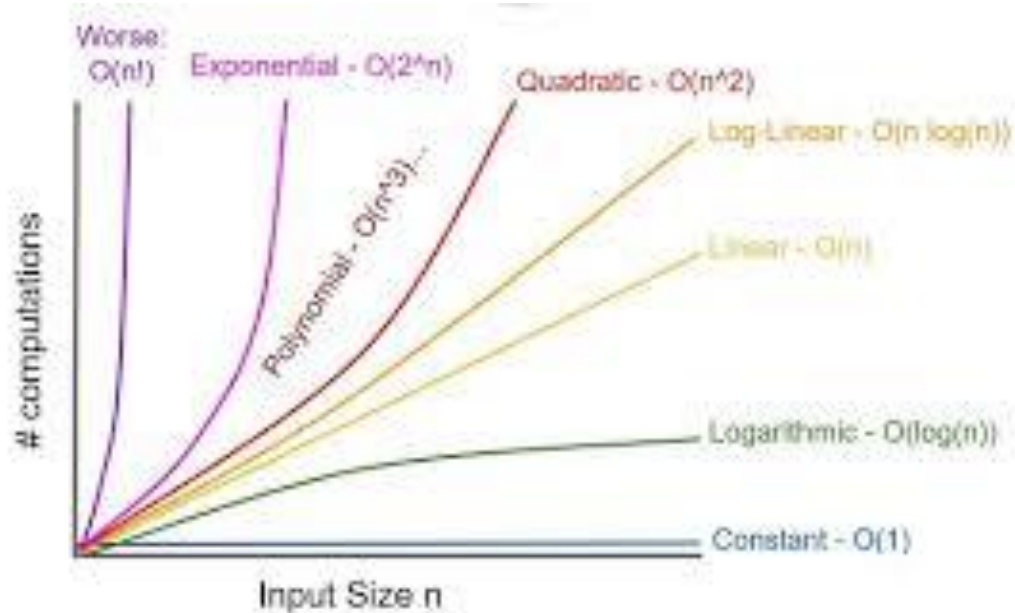
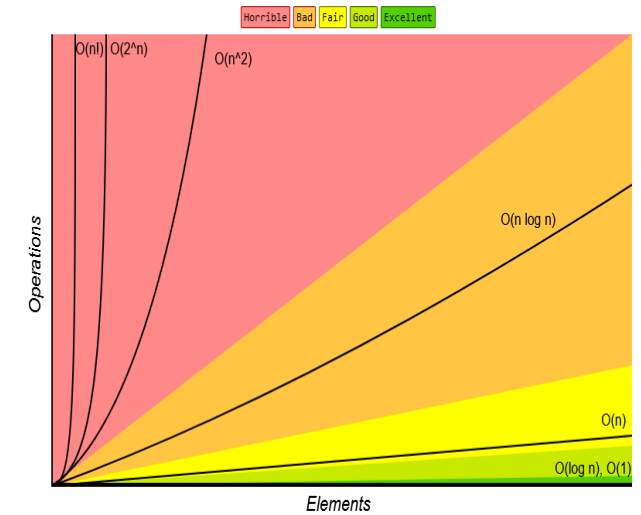
Sequential access



Random access

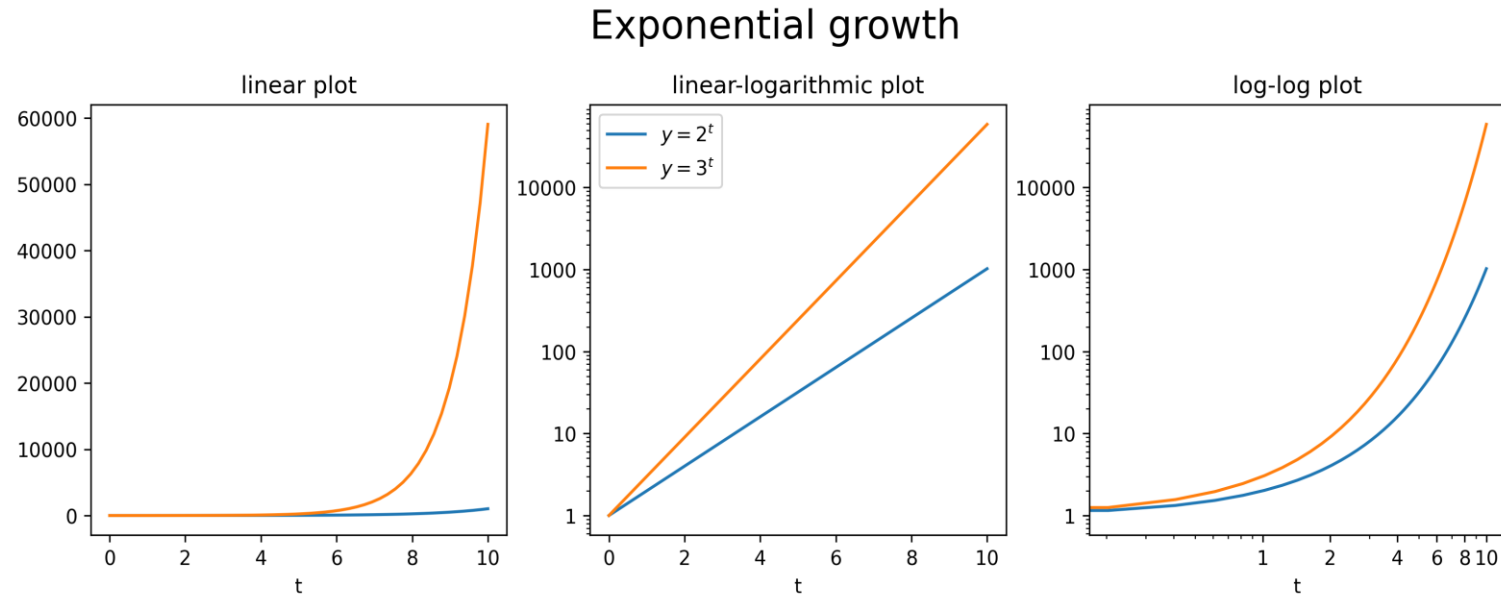
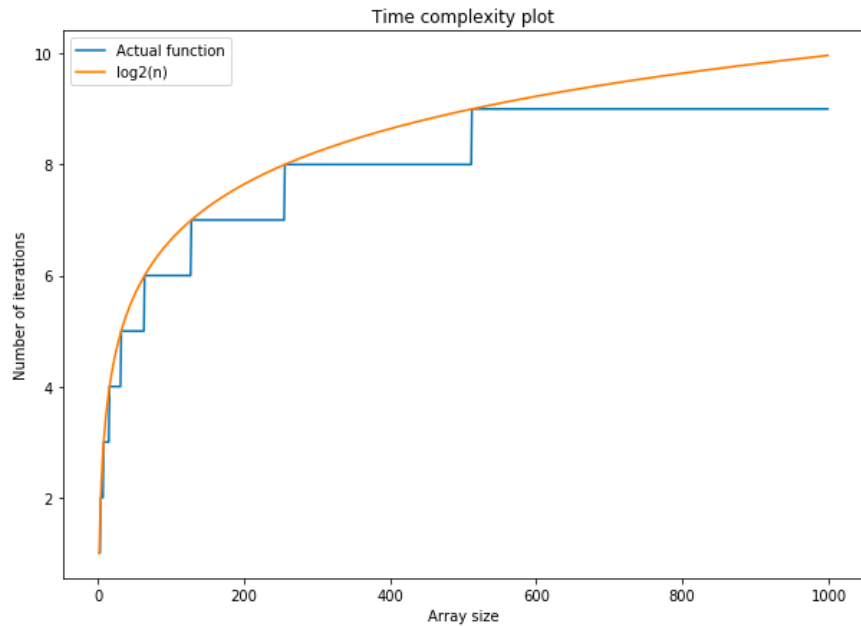


Big-O Complexity Chart

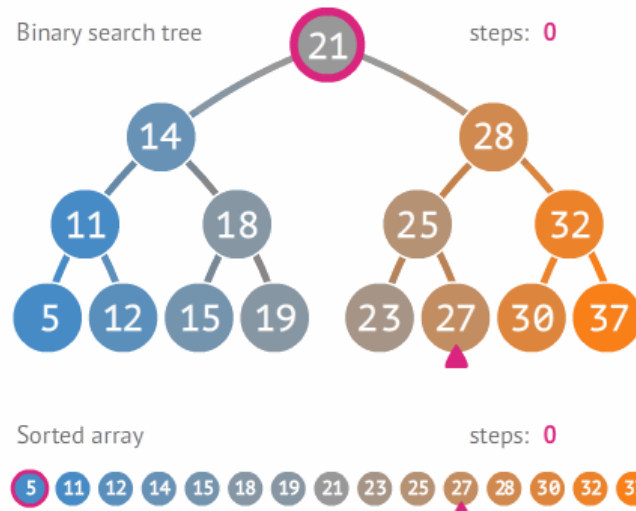
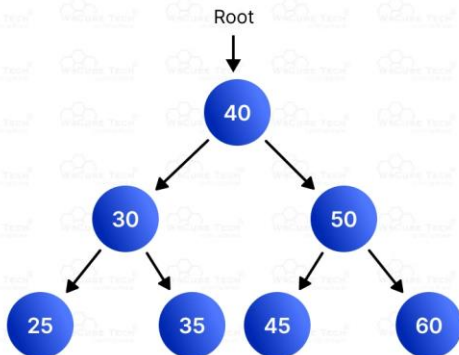


Meaning: Time does NOT depend on input size
Example: Accessing an array element
Always fast

$O(\log N)$ — Logarithmic Time



Binary Search Tree

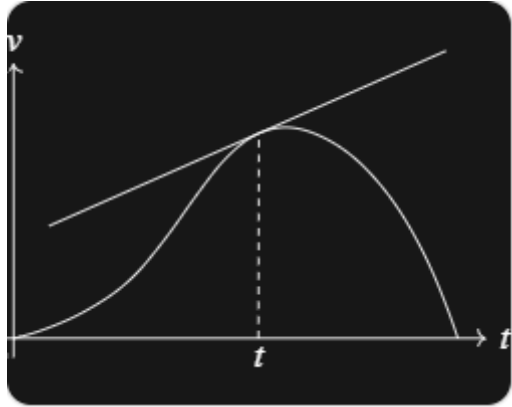


Meaning: Problem size reduces by half each step

Example: Binary Search

Very efficient even for huge data

$O(N)$ — Linear Time



Search 19

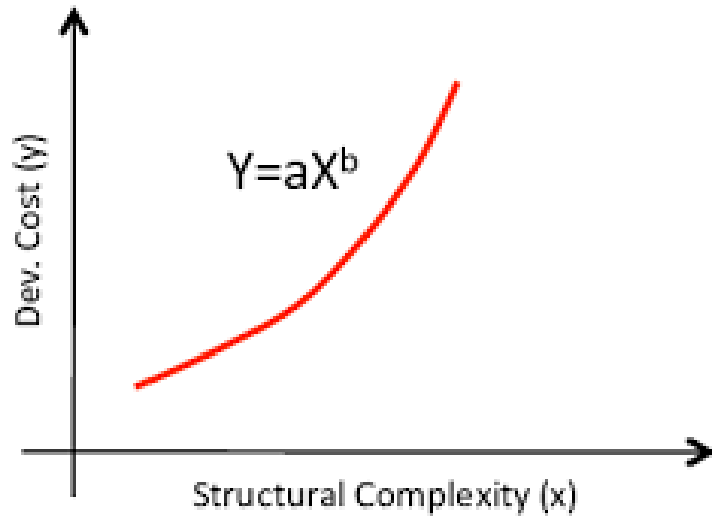
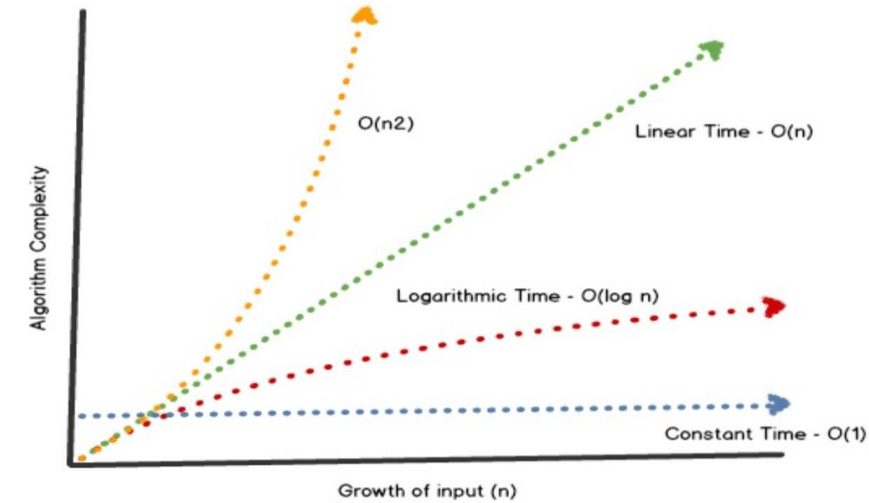
Iteration 1
57 != 19

Iteration 2
69 != 19

Iteration 3
26 != 19

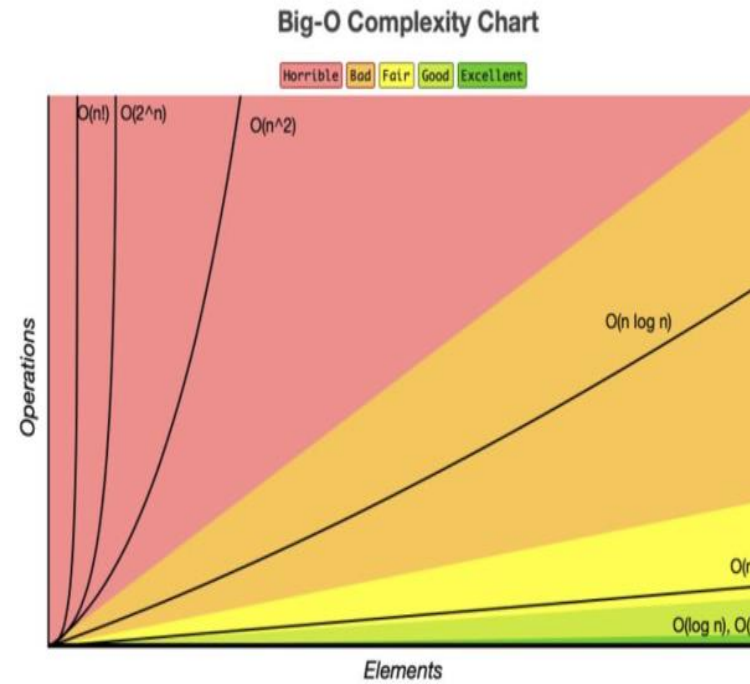
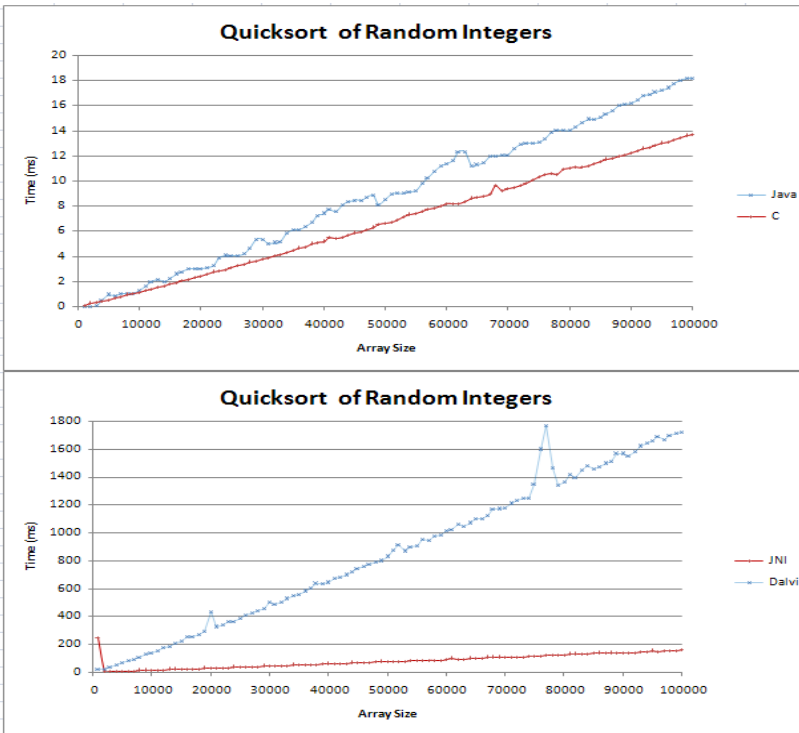
Iteration 4
19 Found

57	69	26	19	28	95	23	18	19	93
57	69	26	19	28	95	23	18	19	93
57	69	26	19	28	95	23	18	19	93
57	69	26	19	28	95	23	18	19	93
57	69	26	19	28	95	23	18	19	93

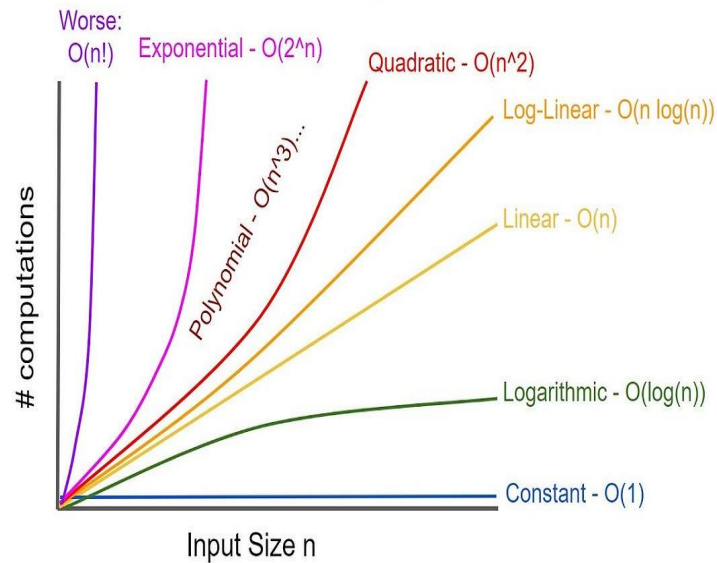


Meaning: Time grows directly with input size
Example: Linear search
Moderate cost

$O(N \log N)$ — Linearithmic

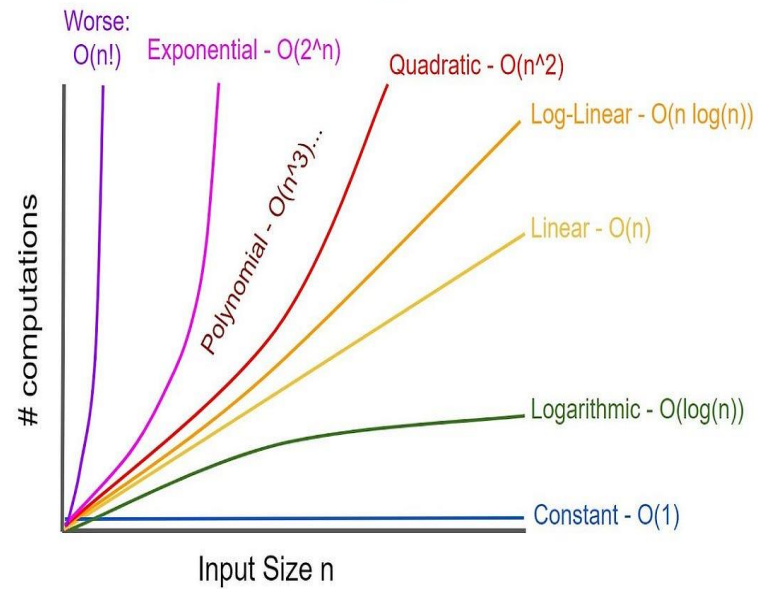
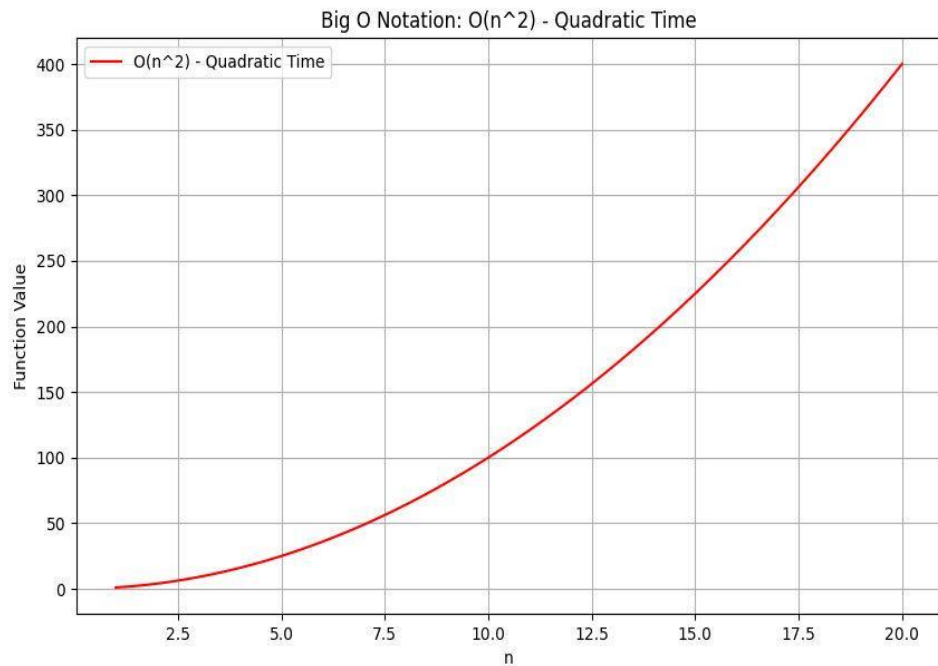
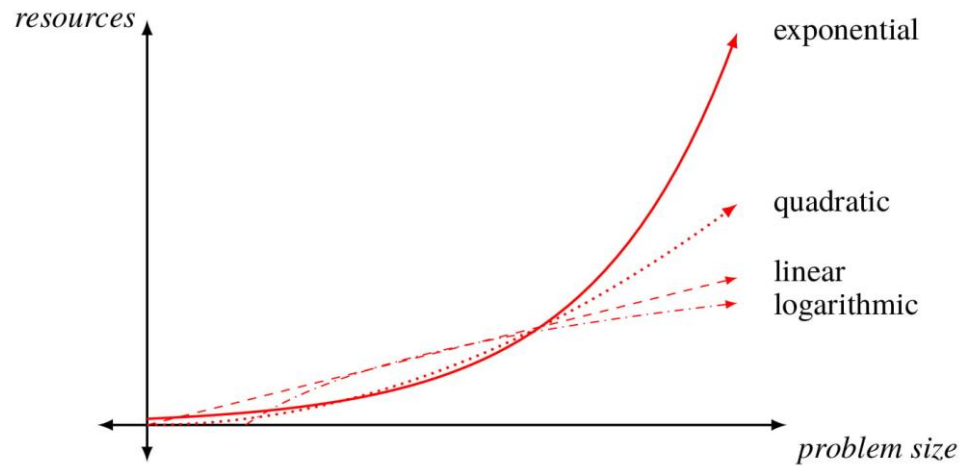


Sorting Algorithms	Time Complexity			Space Complexity
	Best Case	Average Case	Worst Case	Worst Case
Bubble Sort	$\Omega(N)$	$\Theta(N^2)$	$O(N^2)$	$O(1)$
Selection Sort	$\Omega(N^2)$	$\Theta(N^2)$	$O(N^2)$	$O(1)$
Insertion Sort	$\Omega(N)$	$\Theta(N^2)$	$O(N^2)$	$O(1)$
Quick Sort	$\Omega(N \log N)$	$\Theta(N \log N)$	$O(N^2)$	$O(N)$
Merge Sort	$\Omega(N \log N)$	$\Theta(N \log N)$	$O(N \log N)$	$O(N)$
Heap Sort	$\Omega(N \log N)$	$\Theta(N \log N)$	$O(N \log N)$	$O(1)$

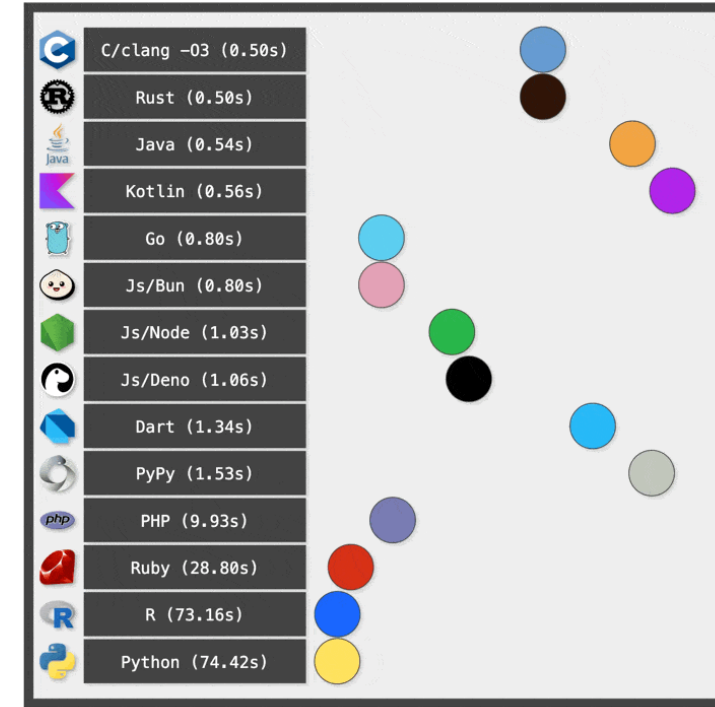


Meaning: Slightly worse than linear
 Example: Quicksort, Mergesort
 Best practical sorting

$O(N^2)$ — Quadratic Time

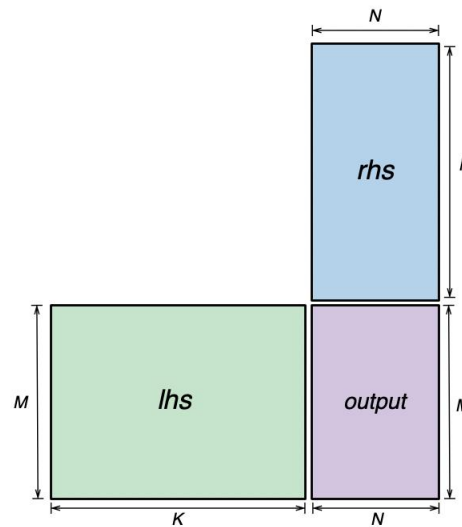
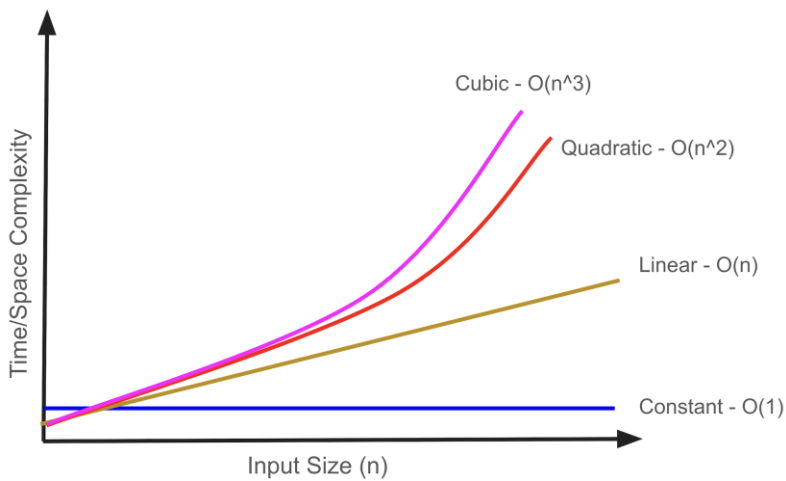


1 Billion nested loop iterations

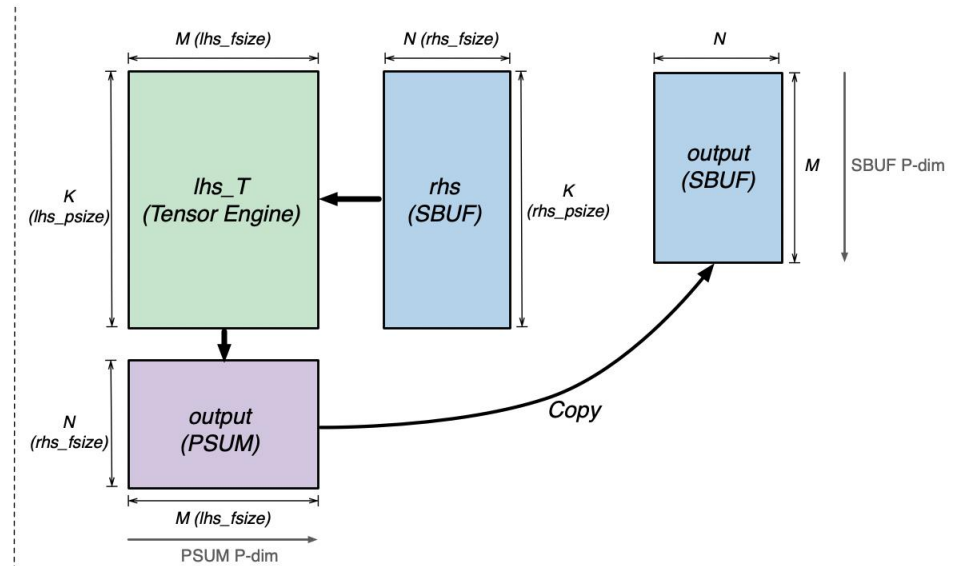


Meaning: Every element compares with every other
Example: Double loops
Becomes slow quickly

$O(N^3)$ — Cubic Time

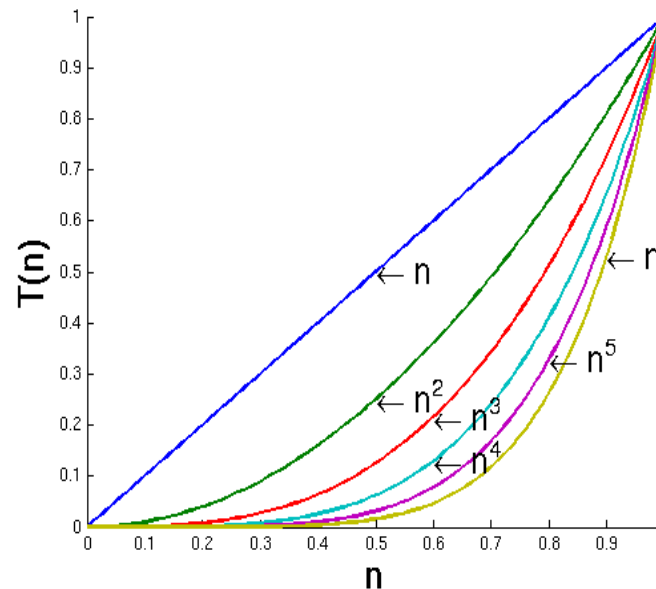
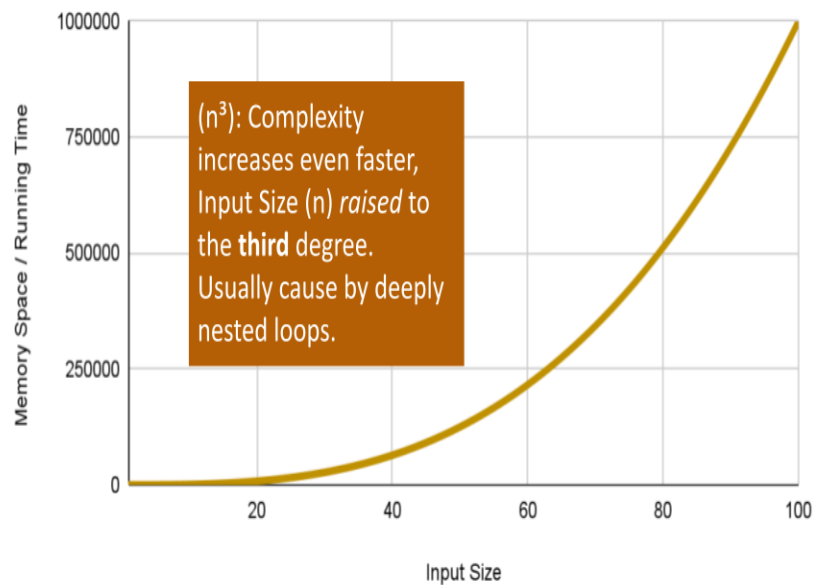


(a) Mathematical View



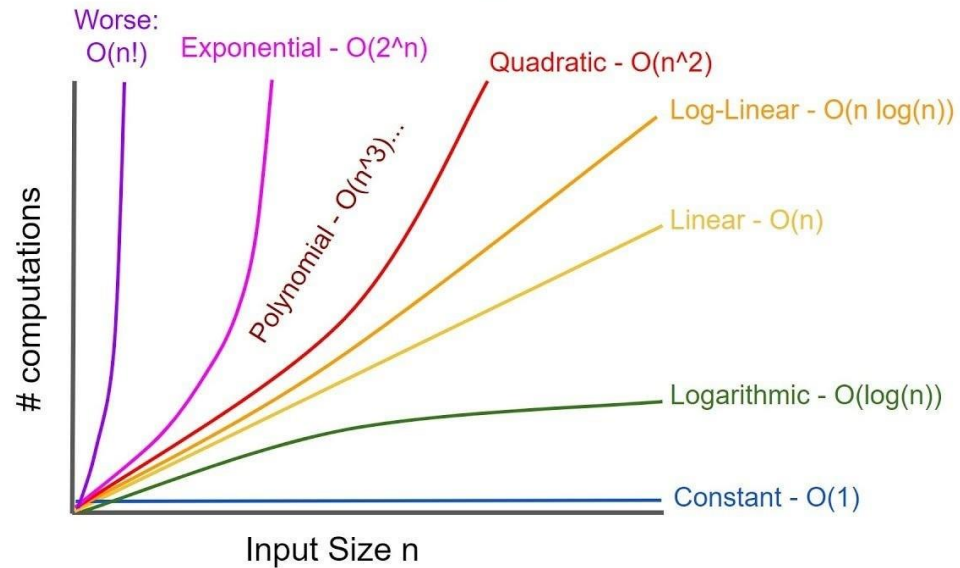
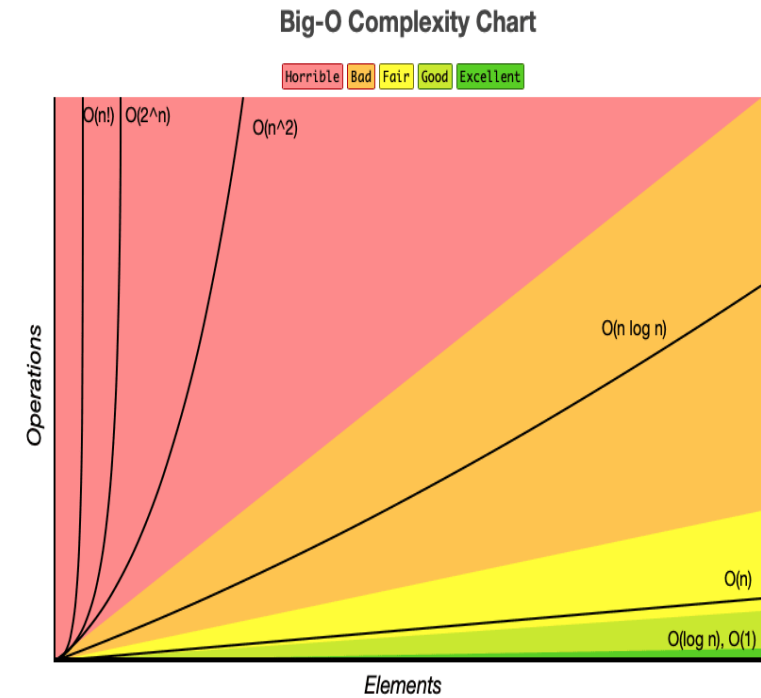
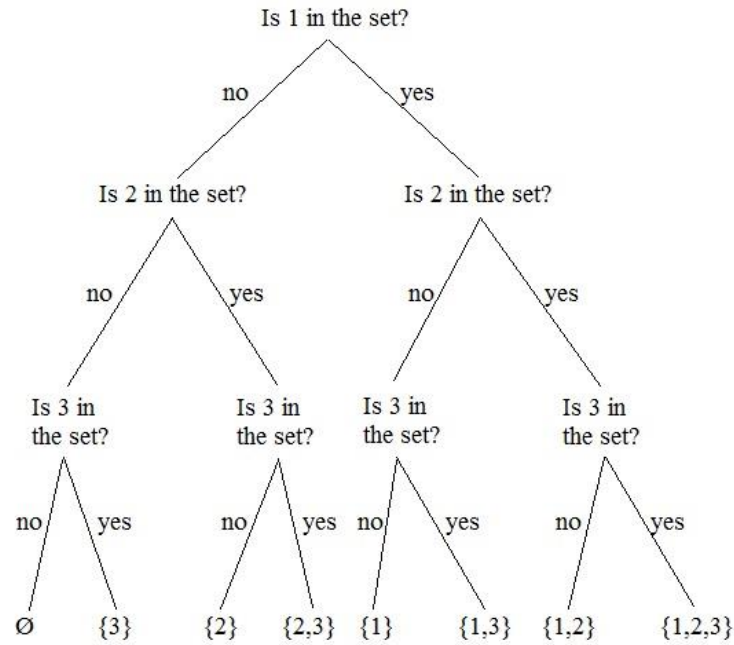
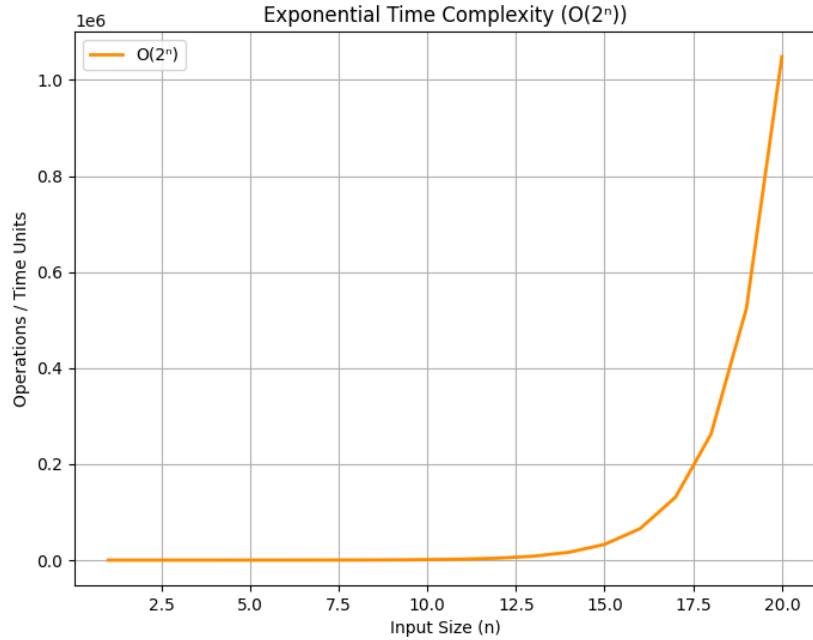
(b) Tensor Engine View

Cubic Complexity Growth



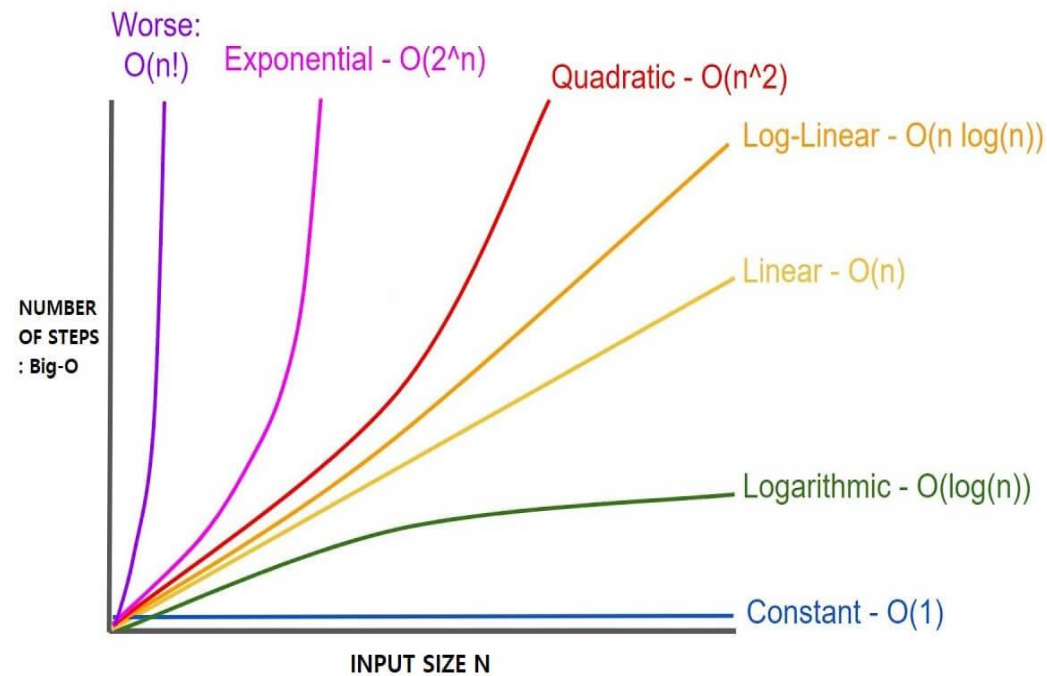
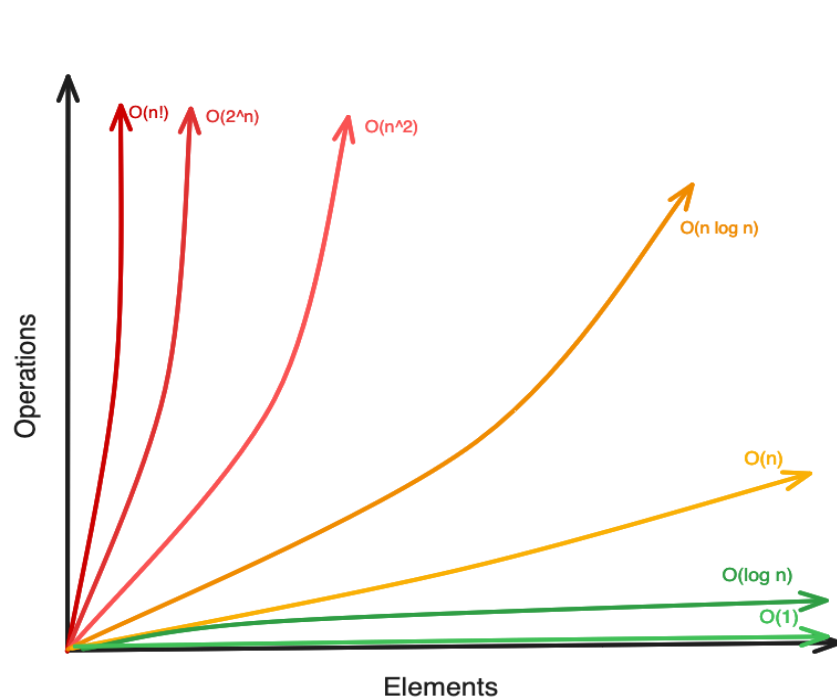
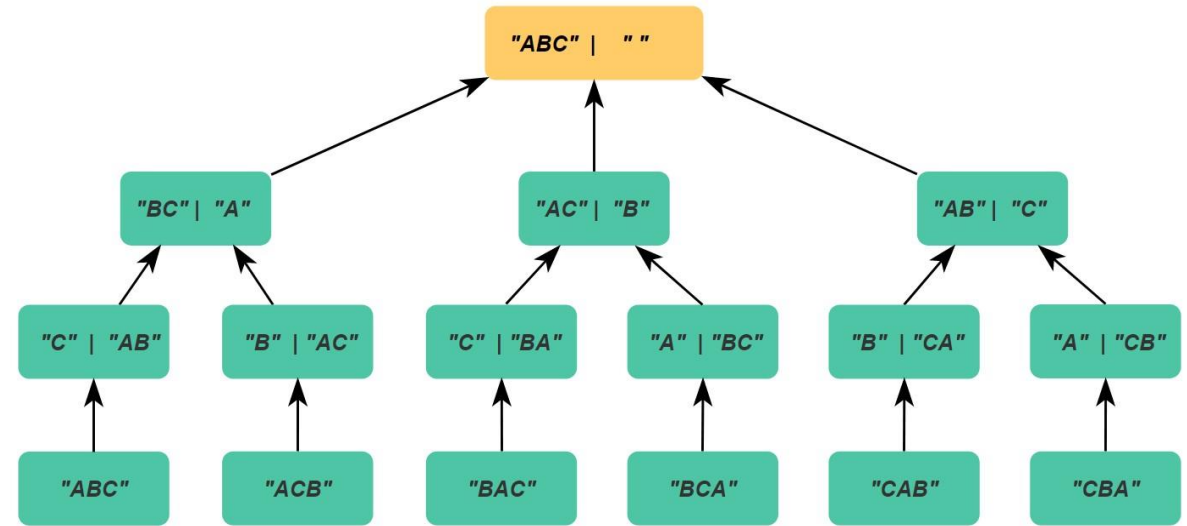
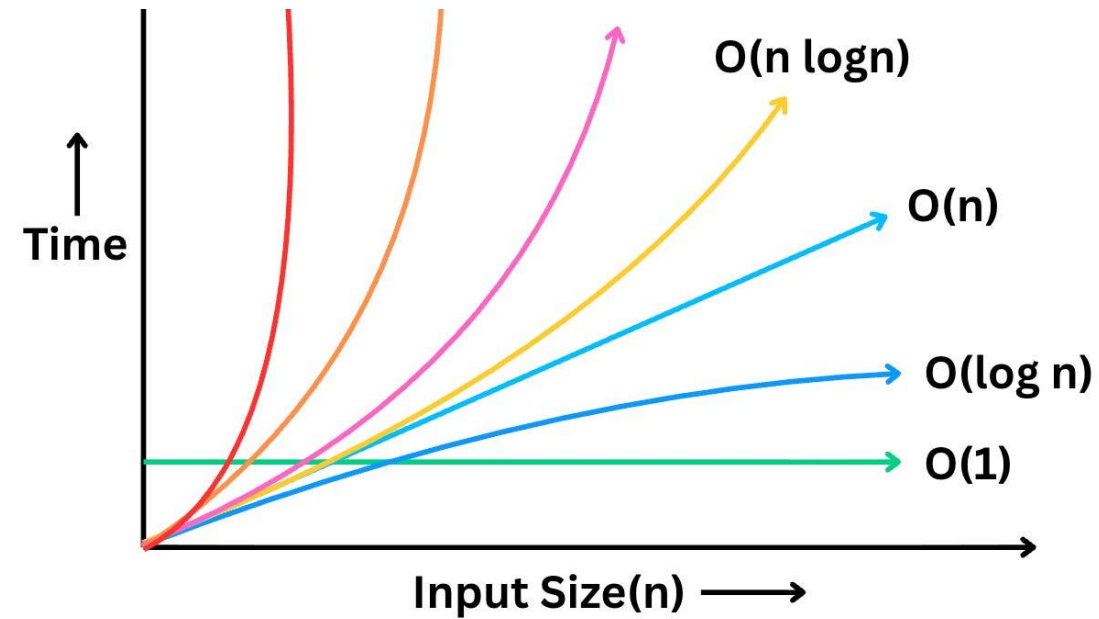
Meaning: Triple nested loops
 Example: Matrix multiplication
 Very expensive

$O(2^N)$ — Exponential Time



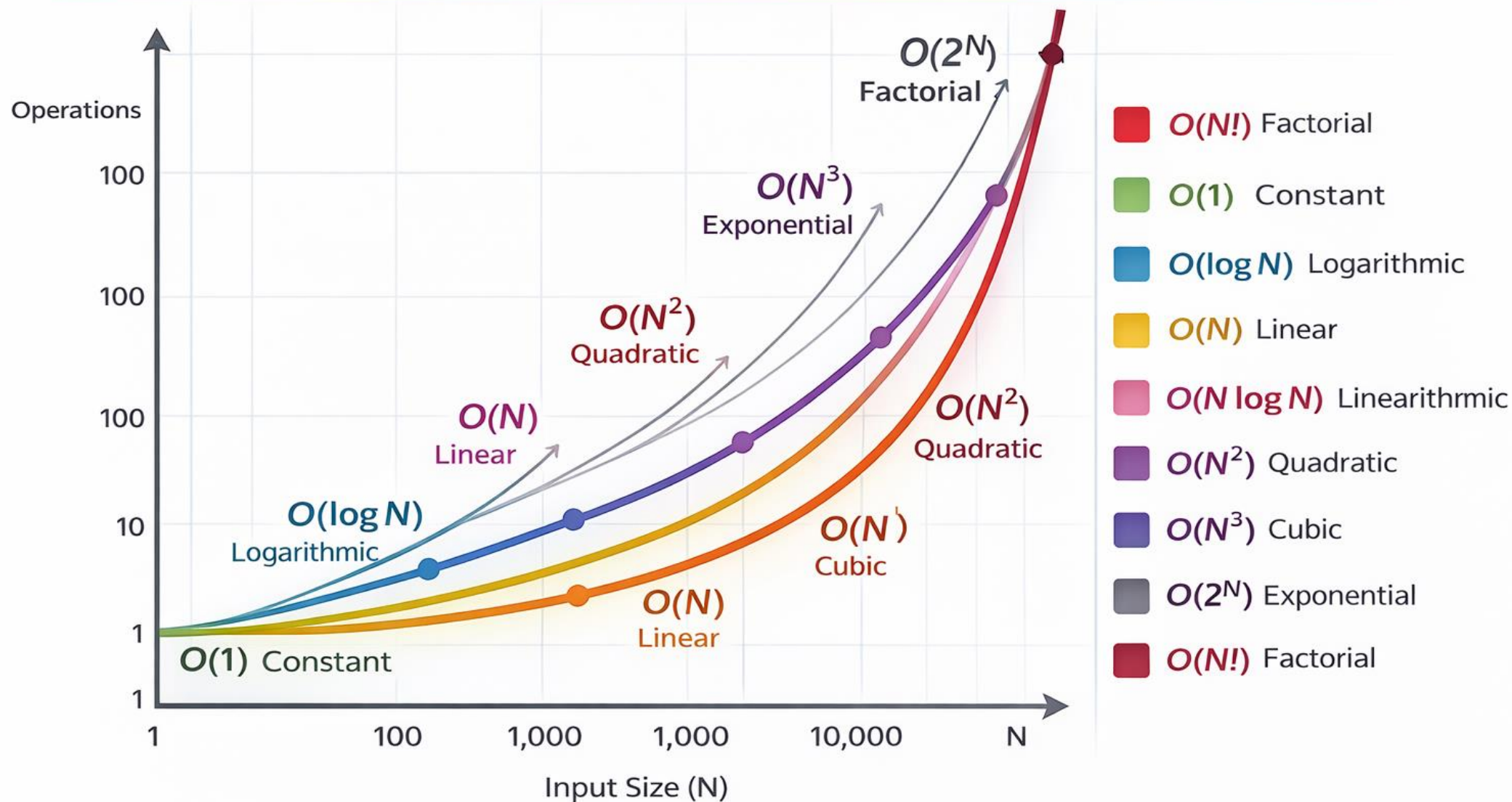
Meaning: Doubles every step
✓ Example: Subsets, brute force
👉 Extremely slow

$O(N!)$ — Factorial Time



Meaning: All permutations
 Example: Traveling salesman brute force
 Practically impossible for large N

Asymptotic Time Complexity



Asymptotic Scaling - Examples

$O(1)$

Array Access



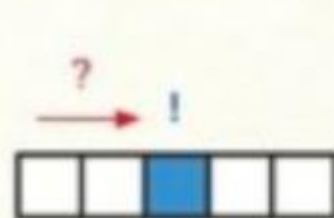
$O(\log N)$

Binary Search



$O(N)$

Linear Search



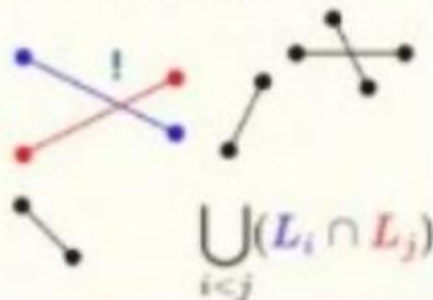
$O(N \log N)$

Quicksort



$O(N^2)$

Comparing Pairs



$O(N^3)$

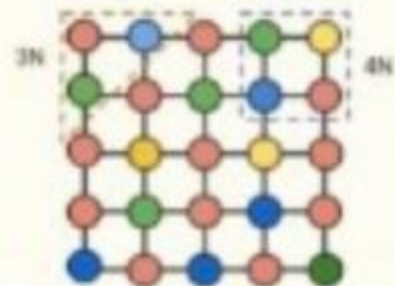
Matrix Multiplication

$$\begin{pmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{pmatrix} \begin{pmatrix} b_{11} & b_{12} & b_{13} \\ b_{21} & b_{22} & b_{23} \\ b_{31} & b_{32} & b_{33} \end{pmatrix} = \begin{pmatrix} c_{11} & c_{12} & c_{13} \\ c_{21} & c_{22} & c_{23} \\ c_{31} & c_{32} & c_{33} \end{pmatrix}$$

$$C_{ij} = \sum_{k=1}^3 A_{ik} B_{kj}, \quad i, j \in \{1, 2, 3\}$$

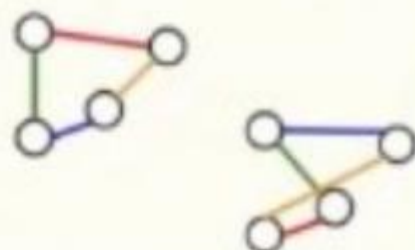
$O(2^N)$

Combinations



$O(N!)$

Permutations



LEGEND

TIME Complexity vs. SPACE Complexity

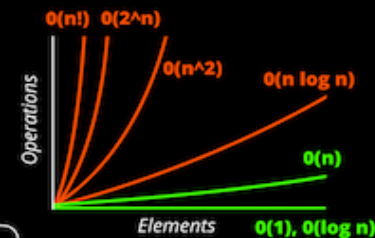
Good Fair Bad

Good Fair Bad

<BIG-O-CHEATSHEET>



www.bigocheatsheet.com



DATA STRUCTURE

Operations

ARRAY SORTING

Algorithms

DATA Structure

TIME Complexity

SPACE Complexity

ARRAY Algorithms

TIME Complexity

SPACE Complexity

Average

Worst

Best

Average

Worst

Worst

		Access	Search	Insertion	Deletion	Access	Search	Insertion	Deletion										
Array		$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$		Quicksort		$\Omega(n \log(n))$	$\Theta(n \log(n))$	$\Theta(n^2)$	$\Theta(\log(n))$		
Stack		$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	$\Theta(1)$	$\Theta(n)$		Mergesort		$\Omega(n \log(n))$	$\Theta(n \log(n))$	$\Theta(n \log(n))$	$\Theta(n)$		
Queue		$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	$\Theta(1)$	$\Theta(n)$		Timsort		$\Omega(n)$	$\Theta(n \log(n))$	$\Theta(n \log(n))$	$\Theta(1)$		
Singly-Linked List		$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	$\Theta(1)$	$\Theta(n)$		Heapsort		$\Omega(n \log(n))$	$\Theta(n \log(n))$	$\Theta(n \log(n))$	$\Theta(1)$		
Doubly-Linked List		$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	$\Theta(1)$	$\Theta(n)$		Bubble Sort		$\Omega(n)$	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(1)$		
Skip List		$\Theta(\log(n))$	$\Theta(\log(n))$	$\Theta(\log(n))$	$\Theta(\log(n))$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n \log(n))$		Insertion Sort		$\Omega(n)$	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(1)$		
Hash Table		N/A	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	N/A	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$		Selection Sort		$\Omega(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(1)$		
Binary Search Tree		$\Theta(\log(n))$	$\Theta(\log(n))$	$\Theta(\log(n))$	$\Theta(\log(n))$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$		Tree Sort		$\Omega(n \log(n))$	$\Theta(n \log(n))$	$\Theta(n^2)$	$\Theta(n)$		
Cartesian Tree		N/A	$\Theta(\log(n))$	$\Theta(\log(n))$	$\Theta(\log(n))$	N/A	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$		Shell Sort		$\Omega(n \log(n))$	$\Theta(n(\log(n))^2)$	$\Theta(n(\log(n))^2)$	$\Theta(1)$		
B-Tree		$\Theta(\log(n))$	$\Theta(\log(n))$	$\Theta(\log(n))$	$\Theta(\log(n))$	$\Theta(\log(n))$	$\Theta(\log(n))$	$\Theta(\log(n))$	$\Theta(\log(n))$	$\Theta(n)$		Bucket Sort		$\Omega(n+k)$	$\Theta(n+k)$	$\Theta(n^2)$	$\Theta(n)$		
Red-Black Tree		$\Theta(\log(n))$	$\Theta(\log(n))$	$\Theta(\log(n))$	$\Theta(\log(n))$	$\Theta(\log(n))$	$\Theta(\log(n))$	$\Theta(\log(n))$	$\Theta(\log(n))$	$\Theta(n)$		Radix Sort		$\Omega(nk)$	$\Theta(nk)$	$\Theta(nk)$	$\Theta(n+k)$		
Splay Tree		N/A	$\Theta(\log(n))$	$\Theta(\log(n))$	$\Theta(\log(n))$	N/A	$\Theta(\log(n))$	$\Theta(\log(n))$	$\Theta(\log(n))$	$\Theta(n)$		Counting Sort		$\Omega(n+k)$	$\Theta(n+k)$	$\Theta(n+k)$	$\Theta(k)$		
AVL Tree		$\Theta(\log(n))$	$\Theta(\log(n))$	$\Theta(\log(n))$	$\Theta(\log(n))$	$\Theta(\log(n))$	$\Theta(\log(n))$	$\Theta(\log(n))$	$\Theta(\log(n))$	$\Theta(n)$		Cubesort		$\Omega(n)$	$\Theta(n \log(n))$	$\Theta(n \log(n))$	$\Theta(n)$		
KD Tree		$\Theta(\log(n))$	$\Theta(\log(n))$	$\Theta(\log(n))$	$\Theta(\log(n))$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$									

$$O(1) < O(\log N) < O(N) < O(N \log N) < O(N^2) < O(N^3) < O(2N) < O(N!)$$